

LoRA

논문 제목

LoRA: Low-Rank Adaptation of Large Language Models

저자

- Edward Hu 외 7인

링크

- <https://arxiv.org/pdf/2106.09685>
-

[Abstract]

모델 규모가 커지면서 모든 파라미터를 재학습하는 full fine-tuning은 비현실적

⇒ Low-Rank Adaptation (LoRA)

사전학습된 모델의 가중치를 고정한 채, Transformer의 각 레이어에 학습 가능한 low decomposition(저랭크) 행렬 주입 → 다운스트림 작업에서 학습해야 하는 파라미터 수 감소

GPT-3 175B를 Adam으로 fine-tuning한 경우와 비교하여, LoRA는 학습 가능한 파라미터 수를 10,000배를 감소시키고 GPU 메모리 요구량 3배 절감 가능

파라미터 수가 훨씬 적지만 fine-tuning과 동등하거나 더 우수한 성능, 더 높은 학습 처리량, adapter 방식과 달리 inference latency(추론 지연) 추가 X

1. Introduction

fine-tuning의 단점: 새 모델이 원본 모델과 동일한 수의 파라미터 가짐

→ 일부 파라미터만 적응시키거나 외부 모듈을 학습하는 방식

→ 한계: 모델 깊이를 증가시키면서 추론 지연 유발 / 사용 가능한 시퀀스 길이 감소 / fine-tuning 대비 성능 저하

과매개변수(over-parameterized) 모델이 실제로는 낮은 intrinsic dimension에 존재한다는 연구들에서 영감

→ 가설: 모델 적응 시 가중치 변화 역시 낮은 intrinsic rank를 가질 것

⇒ LoRA: 사전학습 가중치를 고정한 채, 가중치 업데이트를 저랭크 행렬 분해 형태로 학습

→ 주요 장점:

- 사전학습 모델을 공유하고 작업별로 작은 LoRA 모듈만 교체 가능
- 학습 시 필요한 메모리 및 연산량 대폭 감소
- 학습된 행렬을 가중치에 병합 가능 → 추론 지연 없음
- 기존 기법들과 직교적(orthogonal) → 결합 가능

<Terminologies>

- d_{model} : Transformer 레이어의 입력 및 출력 차원 크기
- W_q, W_k, W_v, W_o : self-attention 모듈에서 query, key, value, output projection 행렬
- W 또는 W_0 : 사전 학습된 가중치 행렬
- 델타 W : Adaptation 중에 누적된 기울기 업데이트
- r : LoRA 모듈의 rank

2. Problem Statement

- 중심 사례: 언어 모델링(Language modeling)

작업별 프롬프트(task-specific prompt)가 주어졌을 때 조건부 확률을 최대화하는 문제

Φ 로 parameterize된 사전 학습된 autoregressive model $P_{\Phi}(y|x)$

→ 다운스트림 조건부 텍스트 생성 task에 사전 학습된 이 모델을 적용, 각 다운스트림 task는 context-target 쌍의 학습 데이터셋으로 표시

$$\mathcal{Z} = \{(x_i, y_i)\}_{i=1, \dots, N}$$

x_i, y_i : 토큰 시퀀스

Full Fine-Tuning

모델을 사전학습 가중치로 초기화한 뒤, 조건부 언어 모델링 목적 함수를 최대화하도록 업데이트

$$\max_{\Phi} \sum_{(x,y) \in Z} \sum_{t=1}^{|y|} \log P_{\Phi}(y_t \mid x, y_{<t})$$

문제점: 각 다운스트림 작업마다 학습되는 파라미터 증가량의 차원이 원래 모델 파라미터 크기와 동일

$$|\Delta\Phi| = |\Phi_0|$$

Parameter-Efficient Adaptation

작업별 파라미터 변화량을 직접 학습하는 대신, 더 작은 파라미터 집합으로 표현

$$\Delta\Phi = \Delta\Phi(\Theta), \quad |\Theta| \ll |\Phi_0|$$

⇒ 최적화:

$$\max_{\Theta} \sum_{(x,y) \in Z} \sum_{t=1}^{|y|} \log P_{\Phi_0 + \Delta\Phi(\Theta)}(y_t \mid x, y_{<t})$$

→ 델타 파라미터를 low-rank 구조로 표현하는 방법 제안

3. Aren't Existing Solutions Good Enough?

1. Adapter Layers 추가 방식

“추론 지연 유발”

- a. 각 transformer block마다 두 개의 adapter layer 사용
- b. block마다 하나의 adapter layer + 추가 LayerNorm

Adapter 방식은 추론 시 추가 연산을 피할 수 없음

Layer를 pruning하여 연산을 줄여, adapter layer의 추가 연산을 우회하는 방법들이 존재하는데 순차적으로 진행되어야 하기 때문에 병렬 활용 불

2. 입력 프롬프트 또는 임베딩 최적화 방식

prefix tuning 같은 방식: 모델 가중치를 수정하지 않고 입력 프롬프트 학습

- 최적화 어려움, 학습 파라미터 수 증가에 대해 성능 단조 증가 X, 원 논문과 유사한 불안정 현상 관찰
- 적응을 위해 시퀀스 길이 일부를 사용해야 하므로 다운스트림 작업에 사용할 수 있는 실제 토큰 길이 감소

4. Our Method

*일반적인 딥러닝 모델의 모든 dense layer에 적용 가능하지만, 본 논문에서는 Transformer 기반 언어 모델에 초점을 맞춤

4.1 Low-Rank-Parametrized Update Matrices

(저랭크 파라미터화 업데이트 행렬)

신경망은 다수의 dense layer 포함, 레이어들은 행렬 곱 연산, 레이어의 가중치 행렬은 일반적으로 full-rank

사전학습 언어 모델은 낮은 intrinsic dimension을 가짐 → 모델 적응 시 가중치 업데이트 역시 낮은 intrinsic rank를 가질 것이라는 가설

- 사전 학습된 가중치 행렬

$$W_0 \in \mathbb{R}^{d \times k}$$

- LoRA의 가중치 업데이트

$$W = W_0 + \Delta W = W_0 + BA$$

$$B \in \mathbb{R}^{d \times r}$$

$$A \in \mathbb{R}^{r \times k}$$

$$r \ll \min(d, k)$$

즉, 업데이트 행렬을 low-rank decomposition 형태로 표현

학습 과정에서 W_0 고정, A, B만 학습

⇒ forward pass:

$$h = W_0x + \Delta Wx = W_0x + BAx$$

- A, B를 random Gaussian initialization, Zero initialization으로 초기화, 학습 초반에는 weight의 변화 값을 0으로 출발

[A Generalization of Full Fine-tuning]

Fine-tuning의 일반화된 버전은 pre-trained parameters의 일부를 학습하는 것 포함

LoRA는 adaptation 과정에서 weight matrices의 full-rank를 필요로 하지 않음. 모든 weight 행렬에 적용하고 biases를 훈련하면 LoRA의 rank를 미리 훈련된 가중치 rank와 맞춰 설정함으로써 전체 fine-tuning의 표현력을 회복할 수 있음

⇒ LoRA의 학습 parameter의 수를 늘리면 fine-tuning 성능과 거의 수렴

[No Additional Inference Latency]

연산 및 저장을 $W = W_0 + BA$ 를 통해 진행하고, inference는 평소처럼 진행 가능

다른 task에 적용할 때는 BA를 빼서 pretrained model의 weight 복구 가능

매우 적은 memory overhead로 매우 빠른 연산을 통해 진행 가능하여 inference 과정에서 추가 latency가 거의 없다고 볼 수 O

4.2 Applying LoRA to Transformer

이론적으로 LoRA는 신경망의 어느 weight subset에든 적용 가능

Transformer에는 self-attention 모듈에 4개의 weight matrices(query, key, vector, output), MLP단에 2개 → query, key, vector를 $d \times d$ single matrix로 처리, 단순화를 위해 downstream 작업에서 attention weight만 조정, MLP attention 고정

[Practical Benefits and Limitation]

memory, storage 사용 절감

large Transformer를 Adam으로 학습할 때 VRAM 사용을 2/3으로 감소

$r=4$ 로 정하고, query와 value projection matrices만 adapt했을 때, checkpoint size가 350GB에서 35MB로 10000배가량 감소

학습 과정에서 적은 GPU 사용, I/O bottleneck 줄임

다른 태스크로 바꿀 때도 적은 양의 LoRA weights만 교체하면 되므로 효과적

(한계) 다른 A와 B를 사용하는 입력을 단일 forward pass에서 처리하는 것이 간단하지 X → 동적으로 LoRA 모듈 선택

5. Empirical Experiments

5.1 Baselines

5.2 RoBERTa base/large

5.3 DeBERTa XXL

5.4 GPT-2 medium/large

5.5 Scaling up to GPT-3 175B

⇒ Model 성능

GLIE에서 RoBERTa와 DeBERTa에 LoRA를 적용했을 때, 적은 학습 parameter만으로 기존 adaptation 방법을 넘어 finetuning에 필적하거나 더 좋은 성능

Model & Method	# Trainable Parameters	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.
RoB _{base} (FT)*	125.0M	87.6	94.8	90.2	63.6	92.8	91.9	78.7	91.2	86.4
RoB _{base} (BitFit)*	0.1M	84.7	93.7	92.7	62.0	91.8	84.0	81.5	90.8	85.2
RoB _{base} (Adpt ^D)*	0.3M	87.1 $\pm$ 0	94.2 $\pm$ 1	88.5 $\pm$ 1.1	60.8 $\pm$ 4	93.1 $\pm$ 1	90.2 $\pm$ 0	71.5 $\pm$ 2.7	89.7 $\pm$ 3	84.4
RoB _{base} (Adpt ^D)*	0.9M	87.3 $\pm$ 1	94.7 $\pm$ 3	88.4 $\pm$ 1	62.6 $\pm$ 9	93.0 $\pm$ 2	90.6 $\pm$ 0	75.9 $\pm$ 2.2	90.3 $\pm$ 1	85.4
RoB _{base} (LoRA)	0.3M	87.5 $\pm$ 3	95.1$\pm$2	89.7 $\pm$ 7	63.4 $\pm$ 1.2	93.3$\pm$3	90.8 $\pm$ 1	86.6$\pm$7	91.5$\pm$2	87.2
RoB _{large} (FT)*	355.0M	90.2	96.4	90.9	68.0	94.7	92.2	86.6	92.4	88.9
RoB _{large} (LoRA)	0.8M	90.6$\pm$2	96.2 $\pm$ 5	90.9$\pm$1.2	68.2$\pm$1.9	94.9$\pm$3	91.6 $\pm$ 1	87.4$\pm$2.5	92.6$\pm$2	89.0
RoB _{large} (Adpt ^P)†	3.0M	90.2 $\pm$ 3	96.1 $\pm$ 3	90.2 $\pm$ 7	68.3$\pm$1.0	94.8$\pm$2	91.9$\pm$1	83.8 $\pm$ 2.9	92.1 $\pm$ 7	88.4
RoB _{large} (Adpt ^P)†	0.8M	90.5$\pm$3	96.6$\pm$2	89.7 $\pm$ 1.2	67.8 $\pm$ 2.5	94.8$\pm$3	91.7 $\pm$ 2	80.1 $\pm$ 2.9	91.9 $\pm$ 4	87.9
RoB _{large} (Adpt ^H)†	6.0M	89.9 $\pm$ 5	96.2 $\pm$ 3	88.7 $\pm$ 2.9	66.5 $\pm$ 4.4	94.7 $\pm$ 2	92.1 $\pm$ 1	83.4 $\pm$ 1.1	91.0 $\pm$ 1.7	87.8
RoB _{large} (Adpt ^H)†	0.8M	90.3 $\pm$ 3	96.3 $\pm$ 5	87.7 $\pm$ 1.7	66.3 $\pm$ 2.0	94.7 $\pm$ 2	91.5 $\pm$ 1	72.9 $\pm$ 2.9	91.5 $\pm$ 5	86.4
RoB _{large} (LoRA)†	0.8M	90.6$\pm$2	96.2 $\pm$ 5	90.2$\pm$1.0	68.2 $\pm$ 1.9	94.8$\pm$3	91.6 $\pm$ 2	85.2$\pm$1.1	92.3$\pm$5	88.6
DeB _{XXL} (FT)*	1500.0M	91.8	97.2	92.0	72.0	96.0	92.7	93.9	92.9	91.1
DeB _{XXL} (LoRA)	4.7M	91.9$\pm$2	96.9 $\pm$ 2	92.6$\pm$6	72.4$\pm$1.1	96.0$\pm$1	92.9$\pm$1	94.9$\pm$4	93.0$\pm$2	91.3

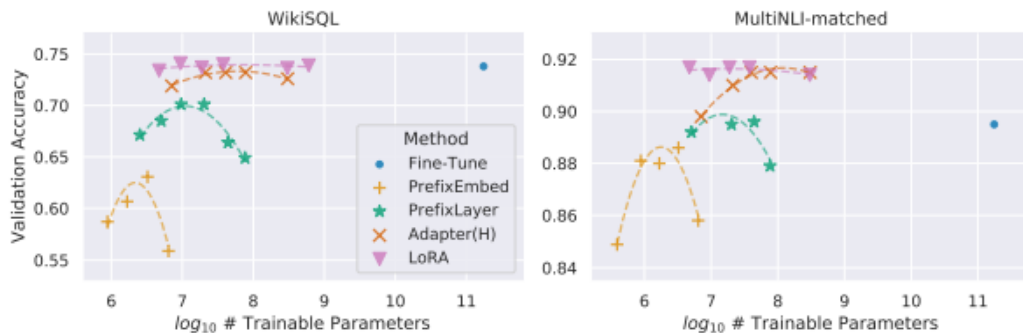
GPT 기반 모델에서 Prefix-layer tuning보다 좋은 성능

Model & Method	# Trainable Parameters	E2E NLG Challenge				
		BLEU	NIST	MET	ROUGE-L	CIDEr
GPT-2 M (FT)*	354.92M	68.2	8.62	46.2	71.0	2.47
GPT-2 M (Adapter ^L)*	0.37M	66.3	8.41	45.0	69.8	2.40
GPT-2 M (Adapter ^L)*	11.09M	68.9	8.71	46.1	71.3	2.47
GPT-2 M (Adapter ^H)	11.09M	67.3 $\pm$ 6	8.50 $\pm$ 0.7	46.0 $\pm$ 2	70.7 $\pm$ 2	2.44 $\pm$ 0.1
GPT-2 M (FT ^{Top2})*	25.19M	68.1	8.59	46.0	70.8	2.41
GPT-2 M (PreLayer)*	0.35M	69.7	8.81	46.1	71.4	2.49
GPT-2 M (LoRA)	0.35M	70.4$\pm$1	8.85$\pm$0.2	46.8$\pm$2	71.8$\pm$1	2.53$\pm$0.2
GPT-2 L (FT)*	774.03M	68.5	8.78	46.0	69.9	2.45
GPT-2 L (Adapter ^L)	0.88M	69.1 $\pm$ 1	8.68 $\pm$ 0.3	46.3 $\pm$ 0	71.4 $\pm$ 2	2.49$\pm$0
GPT-2 L (Adapter ^L)	23.00M	68.9 $\pm$ 3	8.70 $\pm$ 0.4	46.1 $\pm$ 1	71.3 $\pm$ 2	2.45 $\pm$ 0.2
GPT-2 L (PreLayer)*	0.77M	70.3	8.85	46.2	71.7	2.47
GPT-2 L (LoRA)	0.77M	70.4$\pm$1	8.89$\pm$0.2	46.8$\pm$2	72.0$\pm$2	2.47 $\pm$ 0.2

Model&Method	# Trainable Parameters	WikiSQL	MNLI-m	SAMSum
		Acc. (%)	Acc. (%)	R1/R2/RL
GPT-3 (FT)	175,255.8M	73.8	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 (Adapter ^H)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 (Adapter ^H)	40.1M	73.2	91.5	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	91.7	53.8/29.8/45.9
GPT-3 (LoRA)	37.7M	74.0	91.6	53.4/29.2/45.1

⇒ 학습 parameter

accuracy와 학습해야 하는 parameter 수 관계: LoRA가 가장 효율적인 적은 parameter 학습만으로 좋은 성능



6. Related Works

Transformer Language Models

Transformer는 self-attention을 핵심 구성 요소로 사용하는 sequence-to-sequence 아키텍처

Transformer 기반 언어 모델인 BERT와 GPT-2으로 일반 도메인 데이터로 사전학습 후 작업별 데이터로 fine-tuning하는 방식이 작업별 데이터만으로 학습하는 것보다 현저한 성능 향상을 제공한다는 새로운 패러다임 형성

Prompt Engineering and Fine-Tuning

GPT-3 175B는 소수의 추가 학습 예제만으로도 동작을 적응시킬 수 있지만, 결과는 입력 프롬프트에 크게 의존

→ prompt engineering / prompt hacking: 모델 성능을 극대화하기 위해 프롬프트를 설계하고 구성하는 경험적 접근법

→ Fine-tuning: 일반 도메인으로 사전학습된 모델을 특정 작업에 맞춰 재학습하는 방법 - GPT-3 175B 규모에서는 거대한 체크포인트 / 사전학습과 동일한 수준의 메모리 요구량 / 높은 하드웨어 진입 장벽의 이유로 기존 방식의 fine-tuning이 어려움

Parameter-Efficient Adaptation

신경망 레이어 사이에 adapter 모듈 삽입 방식:

- Adapter 레이어: 추론 시 추가 연산 필요
- LoRA: 학습된 가중치를 기존 가중치에 병합하여 추론 지연 없음

모델 가중치 대신 입력 임베딩을 최적화하는 방식:

프롬프트 엔지니어링의 연속적, 미분 가능 일반화

→ 단점: 특수 토큰 수 증가로만 확장 가능

positional embedding 사용 시 시퀀스 길이 감소

Low-Rank Structures in Deep Learning

과매개변수 신경망에서도 학습 이후 저랭크 특성이 나타난다는 연구들이 존재

일부 연구들은 학습 과정에서 명시적으로 저랭크 제약 도입

// 고정된 사전학습 모델에 대해 저랭크 업데이터만 학습하는 적용 방식은 다루지 않음

7. Conclusion and Future work

LoRA의 특성:

- 추론 지연 발생 X
- 입력 시퀀스 길이 감소 X
- 높은 모델 품질 유지
- 사전학습 가중치하며 작업 전환 비용 크게 감소